

Homework 1: Build the Foundation for Your Deep Learning Application

Due: 11:59 PM Eastern Time, September 21, 2026 (No Extension Allowed)

1. Overview

Throughout this course, you will gradually build a custom deep learning application around your own research data or area of interest. Rather than working on completely separate exercises for each homework, you will develop your application incrementally over the course of the semester. Each homework will add a new capability to the tool, eventually resulting in a research-oriented application that uses deep neural networks to perform meaningful tasks on your data. *This homework is the first step towards that.*

That being said, your goal is to build a simple web-based data viewer that will serve as the foundation for your application. At this stage, the viewer does not need to use deep learning. It simply needs to provide a useful interface for viewing and exploring the type of data you expect to work with throughout the semester.

For example:

- If you work with images, start with an image viewer. Later in the semester, you might add image classification, object detection, semantic segmentation, instance segmentation, image generation, or other deep learning capabilities.
- If you work with videos, start with a video viewer. Later, you might add action recognition, object tracking, event detection, or video classification.
- If you work with geographical data, start with an interactive map. Later, you might add prediction, segmentation, clustering, or other learned models.
- If you work with 3D models or meshes, start with a 3D mesh viewer. Later, you might add classification, segmentation, reconstruction, generation, or geometric analysis.
- If you work with audio, start with a waveform/spectrum viewer. Later, you might add speech recognition, sound classification, source separation, or other audio models.
- If you work with social networks, start with an interactive graph viewer. Later, you might add node classification, link prediction, graph embeddings, or graph generation.
- If you work with medical images, start with a slice/volume viewer. Later, you might add segmentation, classification, detection, registration, or other medical imaging models.
- If you work with scientific simulation data, start with an interactive visualization of your simulation results. Later, you might add surrogate modeling, prediction, anomaly detection, parameter estimation, or other deep learning capabilities.
- If you work with time-series or sensor data, start with an interactive signal viewer. Later, you might add forecasting, anomaly detection, event detection, or representation learning.

The specific application is up to you. Choose something that you can continue developing throughout the semester (see below). The viewer you build for this homework should therefore be thought of as the “home” for your future deep learning models. In later assignments, you will progressively add deep learning functionality to this application.

By the end of the semester, the goal is for you to have something substantially more interesting than the simple viewer you build here: a customized research tool that combines your data, an interactive user interface, and deep neural networks.

2. What You Should Think About Now

Before you start coding, think about what your application might eventually become.

For example:

HW1: Image viewer



HW2: Image classification



HW3: Object detection



HW4: Instance segmentation



Final: Interactive image analysis tool

... or:

HW1: 3D mesh viewer



HW2: 3D classification



HW3: Point/mesh segmentation



HW4: Generative model



Final: Interactive 3D deep learning application

You do NOT need to know exactly what your final application will look like. Your application can change as you learn more during the semester. The important thing is to establish a useful starting point now. That said, you should choose a data type and initial viewer that give you room to expand. I suggest that you

speak with your research advisor to sketch out a plan. I will be happy to brainstorm with you as well. You should also think about the availability of data, as training of a deep learning model usually requires a large number of training samples.

3. Web Application Requirement

Your viewer must be implemented as a **web application**. If you already have experience with a frontend framework such as React.js, Vue, Angular, Svelte, or Flutter, you are welcome to use the technology you are familiar with. If you do not have frontend development experience, **React.js** is recommended.

(*Recommended activity*: Search the internet/ask an LLM to learn what React.js is, why it is useful, what you can do with it, what famous websites/applications were built with React.js, etc.)

You are not expected to spend the semester learning frontend programming from scratch. So, if you are not familiar with web development, do not worry about setting everything up manually. You can **ask an LLM or coding agent** to guide you through the process, and you are encouraged to use them to help you build the application. The purpose of this homework is to get your research application started, not to test whether you can memorize JavaScript syntax.

The prompts below are examples. You can copy and paste them into ChatGPT, Claude, Cursor, GitHub Copilot, or another coding assistant and then follow the instructions it gives you.

3.1. Check Whether Your Computer Is Ready

Start here if you don't know whether you have the necessary software installed:

I need to build a simple React.js web application for a class assignment. I am a beginner and have very little web development experience. I am using [Windows/macOS/Linux]. Please help me determine whether my computer has everything I need to create and run a React application locally. Check for Node.js, npm, Git, and anything else that is necessary. Give me one step at a time. For each step, tell me exactly what I should type into the terminal and what I should expect to see. If something is not installed, explain how to install it.

3.2. Create Your React Project

Once Node.js is working:

I have Node.js and npm installed. I want to create a new React.js web application for a class assignment. Please help me create a minimal React project using Vite. My project should be called "[PROJECT NAME]". Give me the commands one at a time and explain where the project files will be created. Then show me how to start the development server and open the application in my web browser.

3.3. Open the Project in an IDE and Make Sure the Basic Application Works

I have created my React project. I am using [VS Code / Cursor / another editor]. Please explain how to open my project folder in the editor and identify the important files I should know about. I am a beginner, so explain what files such as package.json, src, and index.html are used for.

I want to make a very simple page that says "[MY PROJECT NAME]" and has a short description of my research data. Please tell me exactly which file I should modify and provide the code I need. Keep the application as simple as possible.

3.4. Put the Project on GitHub

Once your application works locally:

I have a working React application on my computer and I need to submit it through GitHub. I am a beginner with Git and GitHub. Please walk me through creating a new GitHub repository and uploading my project to it. Explain each step and tell me exactly what commands to run. Please also explain anything I should NOT upload to GitHub, such as passwords, API keys, or large private datasets.

If your data is large or private, ask:

My research data cannot be publicly uploaded to GitHub. Please show me how to structure my project so that the application works with a small sample dataset while keeping the full dataset private.

3.5. Now Build the Viewer

Once your development environment is working, describe what you want to build.

For example:

My React application is now working. I want to turn it into an image viewer for my research data.

I have a folder in [directory] containing approximately 50 PNG images.

I want the application to:

1. Display a list of images
2. Allow the user to select an image
3. Display the selected image in a large viewing area
4. Allow zooming and panning
5. Display the filename

Please implement this one feature at a time and explain what you are changing. Keep the code simple because I will add deep learning functionality to this application in later assignments.

For a different type of research data, simply change the description. Below are examples (add details as you see fit.)

Example – Time Series:

My research data consists of time-series measurements stored in CSV files. Build a React-based viewer that allows me to select a CSV file and display an interactive plot. The user should be able to zoom, pan, and select different variables.

Example – 3D Models:

My research data consists of 3D models in STL format. Build a React web application that displays an interactive 3D model. Use Three.js for rendering. The user should be able to rotate, pan, and zoom the model.

Example – Geographical Data:

My research data consists of geographic coordinates and measurements. Build a React web application with an interactive map. Display my data as markers and show the associated measurements when a marker is selected.

Example – Audio:

My research data consists of WAV audio files. Build a React web application that allows the user to select an audio file, play it, and display its waveform. If possible, also display a frequency spectrum.

Example – Video:

My research data consists of MP4 videos. Build a React web application that allows the user to select a video, play and pause it, move through the timeline, and display basic metadata about the video.

3.6. *If Something Goes Wrong*

Do not immediately start changing random files or reinstalling everything. Instead, give the AI agent the exact error message. Copying the complete error message is usually much more useful than saying “it doesn't work.”

For example:

I followed your instructions and now I get the following error:

[PASTE THE COMPLETE ERROR MESSAGE HERE]

I am a beginner. Please explain what caused this error and tell me the smallest change I need to make to fix it. Do not rewrite the entire project unless necessary.

***** One Important Rule *****

Don't let the AI do everything without understanding it. You do not need to understand every line of JavaScript, but you should understand:

- What your application does
- What the major components are

- Where your data comes from
- How the user interacts with the application
- What libraries you are using
- How to start the application
- How you would ask an AI agent to modify it

The goal is to use AI to lower the barrier to software development, so that you can focus your time on building an interesting deep learning application around your research. However, the goal is never to let AI do all the work for you—how much of a grasp you have on your project is going to determine the quality of the outcome.

4. Recommended Development Strategy

Your first version should be simple. For example, an image viewer might initially have only:

- A list of images
- An image display area
- Zoom/pan
- Basic metadata

..., which is enough for HW1. Later, you can add:

- A "Classify" button
- Predicted labels and confidence scores
- Bounding boxes
- Segmentation masks
- Model selection
- Visualization of intermediate features
- Comparison of model predictions
- User annotations
- Backend storage

The point is to start with a working application and gradually make it smarter. Do not try to build your final application for HW1 (although, if you are itchy, you are not prohibited from making some headstart).

Build your application in small steps:

Step 1 — Get the web application running

Create a minimal React application and make sure you can open it in your browser.

Step 2 — Display your data

Load a small representative dataset and display it.

Step 3 — Add basic interaction

Allow the user to select, navigate, zoom, pan, play, rotate, filter, or otherwise explore the data.

Step 4 — Make the viewer useful

Add the information and controls that you think will be useful for your eventual research application.

Step 5 — Think ahead

Consider where your deep learning models will eventually fit into the interface.

For HW1, Steps 1–4 are the primary objective. Step 5 is there to help you make a good choice of project.

5. Grading

Component	Description	Points
Functional data viewer	Successful submission of a minimally functional data viewer that allows a user to view and interact with your research data in an appropriate way.	+80
Late submission	Penalty for late submission.	-5 per day
Interactive data annotation	Earn additional 10 points by implementing an intuitive mechanism to allow user to annotate/tag/label data. Examples include: bounding boxes; points; polygons; text labels; regions of interest; video timestamps; events in a time series; markers on a map; labels on nodes in a graph; markers on a 3D model.	+10
Backend/database integration	You can earn an additional 10 points by connecting your application to a backend database (can just be a scaffold now; doesn't have to be fully populated). For example, you might use the database to store: annotations; dataset metadata; user selections; analysis results; model predictions. If you are not familiar with backend development, the recommended starting point is "FastAPI + SQLite + Docker" running locally—give those keywords to LLM/Agents and ask for details.	+10

6. Submission

Submit the following through the Canvas assignment Dropbox:

A. Three-Minute Demo Video

Submit a **maximum 3-minute** video showing a live demonstration of your application. Your video should briefly show: (1) what type of data you are working with; (2) your web application; (3) how the viewer works; and (4) the main interactions you implemented. If you are submitting for additional credit, the video must also demonstrate the annotation and/or database functionality.

B. GitHub Repository

Submit a link to a GitHub repository containing your source code. Include a 'README.md' with:

- A brief description of your application
- The type of data being visualized
- Instructions for running the application
- Major libraries/frameworks used
- A brief description of implemented features
- Whether you implemented annotation
- Whether you implemented a backend/database

Disclaimer: ChatGPT was used for proofreading.